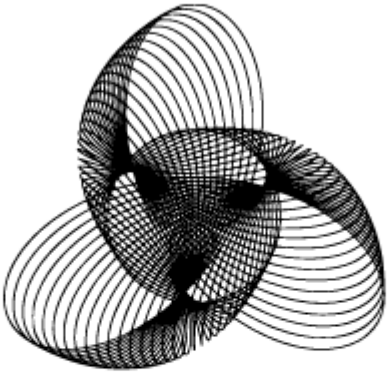


DELTA

MAX 2000

Rapport de troisième Soutenance

Rubix-2000

Theodore Deguest, Arthur Bonnefont, Timothé Le Bronec, Hamza Hannat

Sommaire

I - Introduction: 2
II - Nomenclature: 3
a - Faces: 3
b - Mouvements: 3
c - Cublet: 4
b - Facelet: 4
III - Détection du cube: 5
a - Utilisation: 5
b - Ressources: 7
c - Technique de traitement d'image: 7
d - Phases: 8
e - Structures de données: 8
f - Algorithme: 9
g - Limites: 13
IV - Simulateur: 14
a - Utilisation: 14
b - Ressources: 16
c - Ray-casting: 16
d - Rotation: 17
V - Solveur: 18
a - Choix de l'algorithme: 18
b - Fonctionnement: 18
c - Les tables: 20
c - Performances: 20
VI - Solution: 22
a - Utilisation: 22
b - Ergonomie: 23
VII - Interface: 24
a - Ressources: 24
b - Organisation: 24
c - Ergonomie: 25
d - Paramètres: 26
VIII - Site Web: 28
VIII - Conclusion: 28
Quelques chiffres: 28

I - Introduction:

Ce rapport est le compte-rendu de notre travail depuis le commencement de ce projet. Nous y aborderons l'avancement réalisé depuis la deuxième soutenance mais également une vue plus globale de **Rubik's 2000**.

Depuis la deuxième soutenance, nous avons travaillé jusqu'à remplir l'ensemble des critères présents dans le cahier des charges. A savoir:

- un programme de **détection de Rubik's Cube** via la caméra facile à prendre en main, ergonomique et nécessitant un minimum d'interactions de l'utilisateur
- un **simulateur de Rubik's Cube** en 3d esthétique, facile à utiliser tout en restant complet
- un **solveur de Rubik's Cube** qui trouve une solution rapidement et en moins de 20 coups de moyenne
- une **affichage de la solution** trouvée intuitif
- une **interface utilisateur complète**, esthétique et ergonomique

Tous les axes cités ont été développés en parallèle permettant une progression continue de notre projet tout en évitant de nombreux problèmes d'inter-compatibilité entre ces différentes parties.

Ci-dessous, le tableau d'avancement de notre projet:

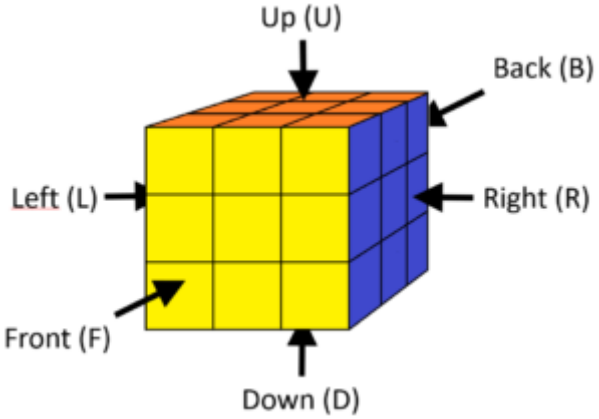
Tâches	**Soutenance 1**	**Soutenance 2**	**Soutenance 3**
3D	20 %	90 %	100 %
Interface utilisateur	0 %	90 %	100 %
Détection du cube	10 %	50 %	100 %
Solveur	0 %	50 %	100 %

II - Nomenclature:

Tout au long de ce rapport, nous utiliserons différents termes appartenant à l’univers du Rubik's Cube. Les définitions de tous ces termes sont regroupées dans cette section.

a - Faces:

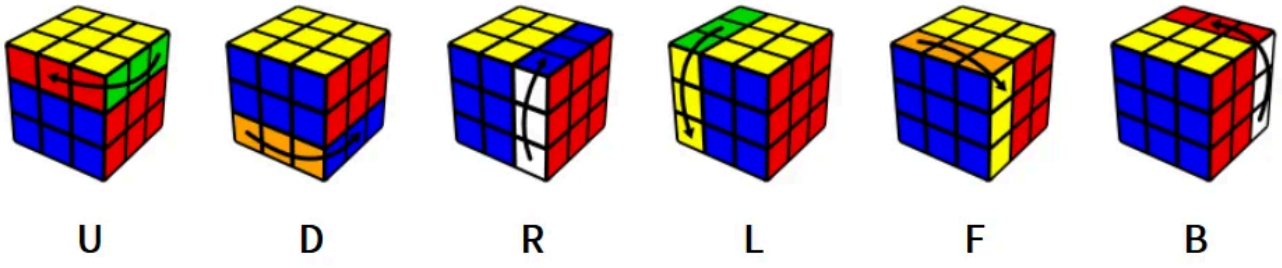
Les faces du Rubik’s Cube sont nommées avec les lettres U, D, R, L, F, et B:



1 - nom des faces

b - Mouvements:

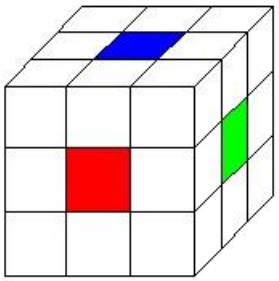
Les également du Rubik’s Cube sont nommées avec les lettres U, D, R, L, F et B en fonction de la face à faire tourner:



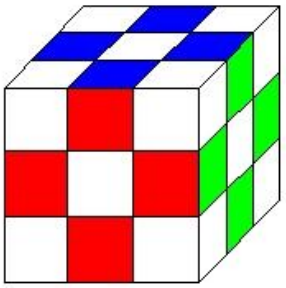
2 - mouvements du cube

c - Cublet:

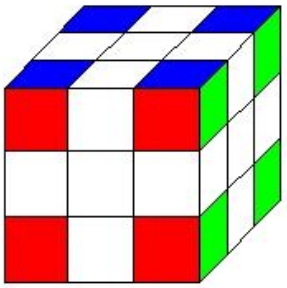
Un Rubik's Cube est composé de 3 x 3 cublets. Un cublet est donc un des sous-cube de ce dernier.



3 - cublets centraux



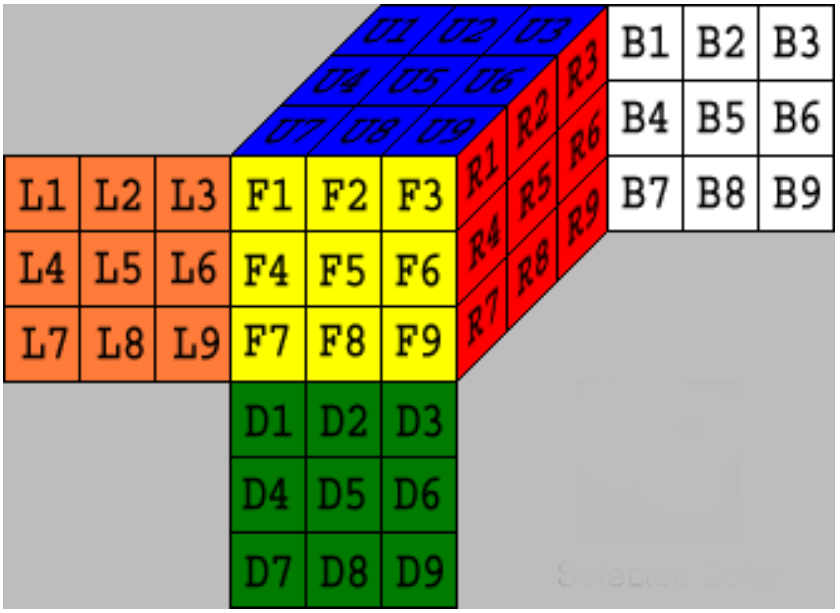
4 - cublets des arêtes



5 - cublets des coins

b - Facelet:

Un Rubik's Cube est composé de 6 * 9 facelets (9 par face). Un facelet est donc une des 54 cases de ce dernier.



6 - facelets d'un cube

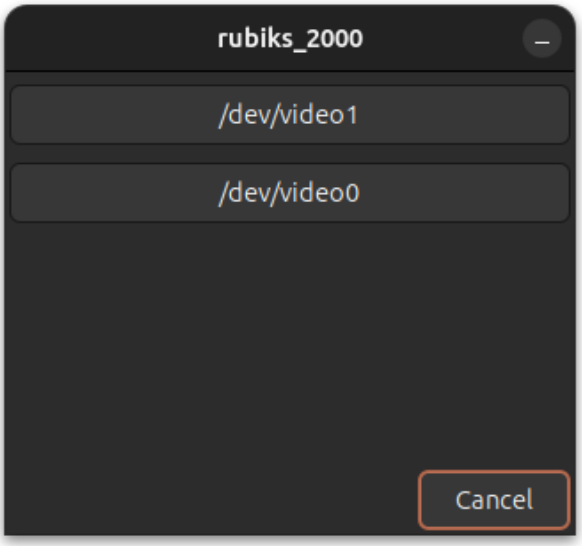
III - Détection du cube:

Cette partie est dédiée à la détection du cube via la caméra. L'objectif de cette fonctionnalité est de fournir à l'utilisateur un moyen simple de transposer son Rubik's Cube (physique) dans notre simulateur. Notre principale problématique a été de trouver un moyen de convertir les données fournies par l'utilisateur lorsqu'il montre son cube à la caméra en une représentation abstraite du cube, lisible par notre simulateur.

a - Utilisation:

Avant de rentrer dans les détails de son fonctionnement, voici un aperçu de son utilisation:

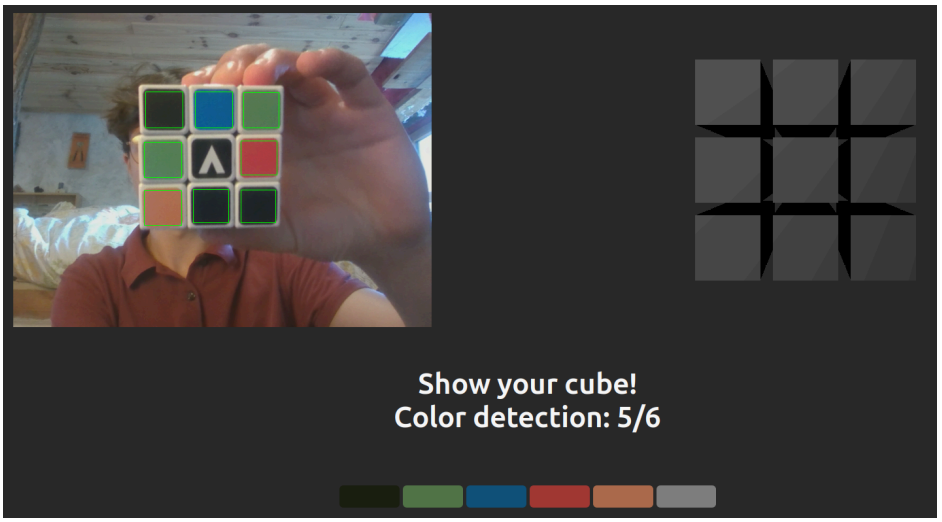
Étape 1: si ça n'est pas encore le cas, sélectionnez la caméra adéquate dans les paramètres. La caméra par défaut est "/dev/video0/".



7 - sélection de la caméra

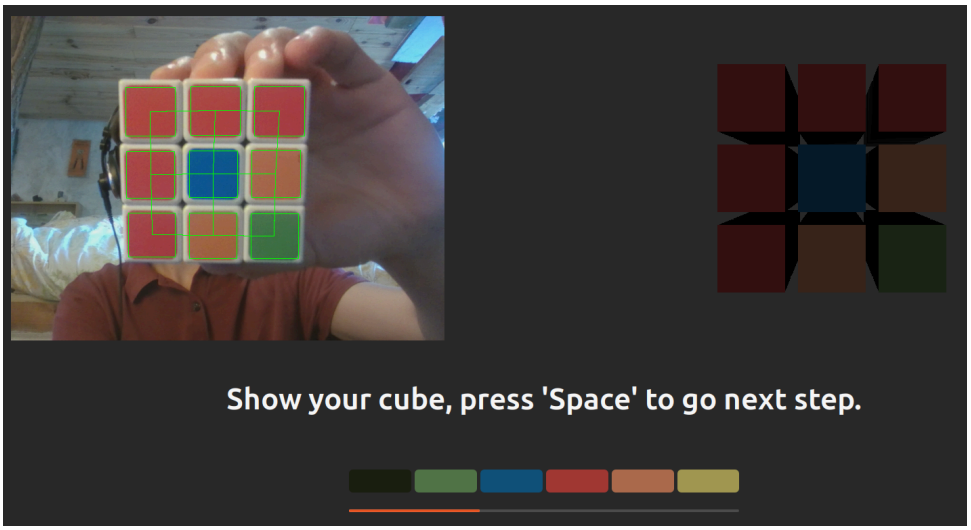
Étape 2: rendez-vous dans la vue de détection du cube.

Étape 3: commencez la calibration des couleurs en montrant votre cube à la caméra. Le programme va alors détecter automatiquement les couleurs présentes sur votre cube et les afficher en bas de l'écran. Si certaines couleurs sont mal détectées ou le sont plusieurs fois, vous pouvez cliquer dessus pour les supprimer.



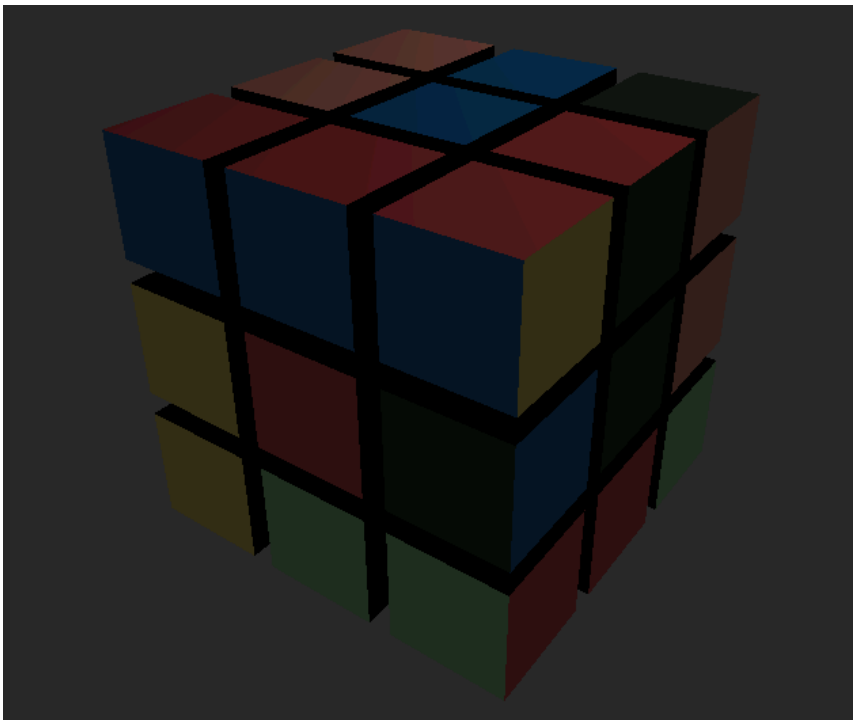
8 - calibrage des couleurs

Étape 4: une fois les 6 couleurs détectées, appuyez sur espace pour commencer la détection des face du cube. Ensuite, suivez les mouvements du cube affiché à l'écran en vérifiant que les données collectées sont exactes. Si une erreur est survenue ou si le programme n'arrive pas à détecter un facelet, vous pouvez cliquer dessus afin de modifier sa couleur.



9 - détection des Facelets

Étape 5: une fois la détection terminée, vous serez redirigé sur la vue du simulateur et votre cube sera chargé automatiquement. Attention, si vous appuyez sur *reset*, toute la détection sera perdue.



10 - cube détecté

Les couleurs affichées sur le cube sont celles qui ont été détectées pour permettre comme ici des couleurs qui ne sont pas dans le cube de base (noir).

b - Ressources:

Afin de capter le flux caméra, nous utilisons la librairie ***rs-cam***, qui nous permet de gérer différent format de caméra ainsi qu'une capture d'image simple et efficace.

c - Technique de traitement d'image:

Il existe de nombreux outils pour atteindre un résultat semblable au nôtre: l'utilisation d'un réseau de neurones, la détection de ligne par transformée de Hough, la détection de contour par filtrages convolutifs, la détection de masses ...

Dans notre cas, nous détectons les différentes formes présentes sur l'image à l'aide d'un algorithme de détection de contour par application du filtre de Sobel couplé à un détection de masse. Puis, nous filtrons ces formes pour ne garder que les carrés à l'aide d'un algorithme de notre confection expliqué plus bas. Enfin, nous créons un graphe des positions des ces différentes formes pour faire correspondre chaque masse détectée à un Facelet du cube.

d - Phases:

Comme vu dans la partie *Utilisation,* il y a deux principales étapes à la détection. La première est le **calibrage de la table des couleurs** et la seconde est la **détection des face et la mise en correspondance**.

Le calibrage de la table des couleurs nous permet de définir les 6 couleurs pour chacune des faces. La détection des faces et la mise en correspondance associe une couleur à chaque Facelet.

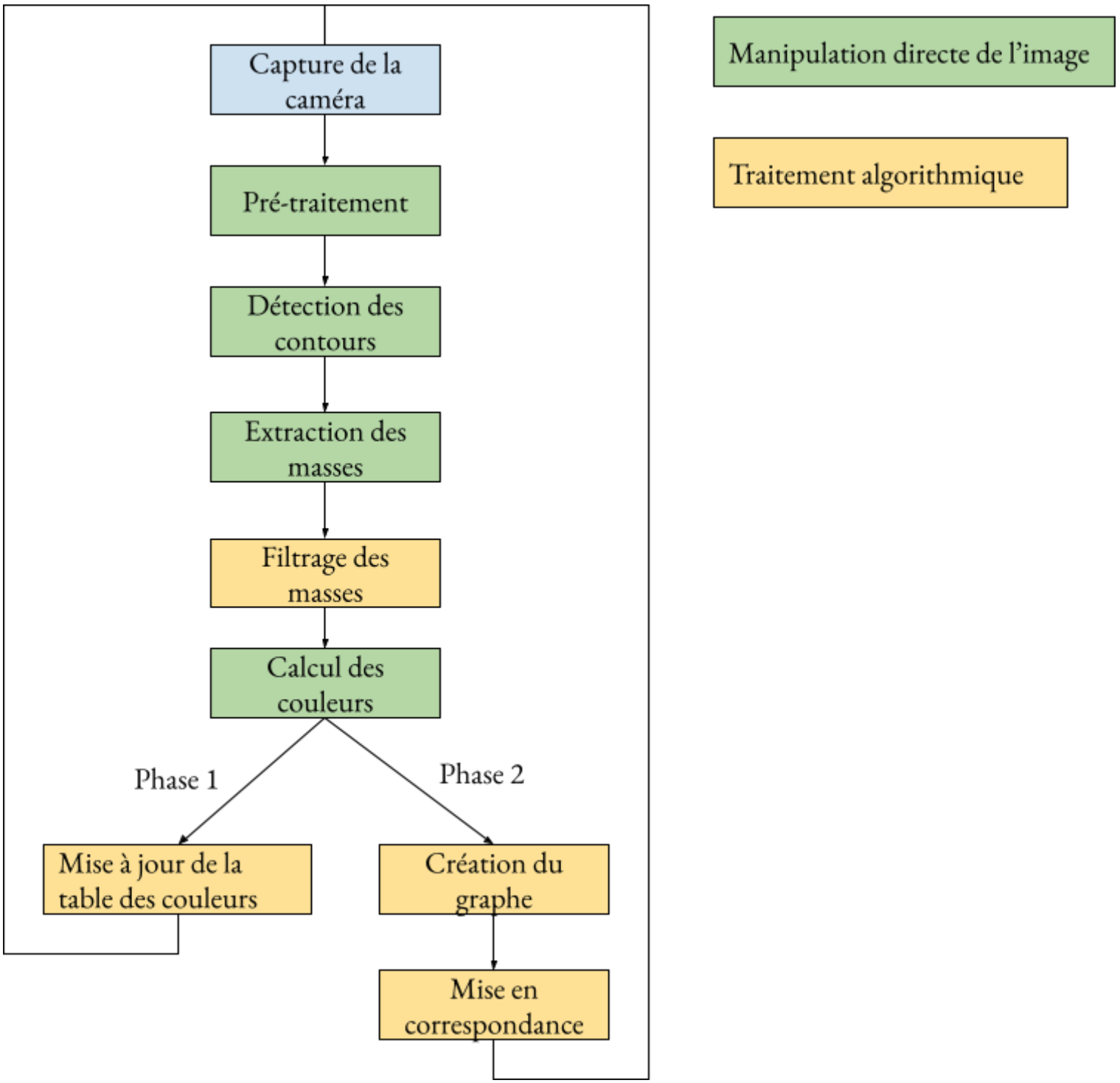
e - Structures de données:

Afin de traiter au mieu l'information capturée, nous utilisons 4 structures de données majeures:

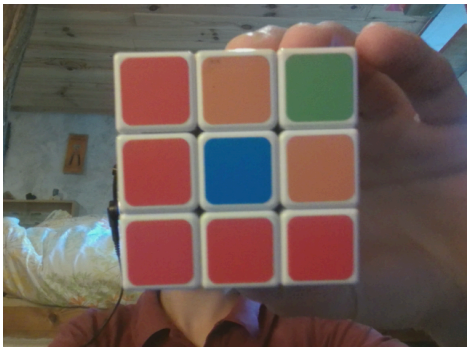
- **Shape** correspond à une masse détectée. Elle contient l'ensemble des couleurs et des positions de ses pixels sous forme de vecteur.
- **ShapeTab** est un tableau de **Shape**, utilisé pour faciliter leur manipulation et pour créer le graphe nous permettant l'association entre les Shapes et les Facelets.
- **ColorTable** est la table de correspondance qui pour chacune des 6 couleurs détectées lors de la phase de calibration associe un FaceTag.
- **RgbData** est la table en deux dimensions qui pour chacun des 54 Facelets du cube associe toutes les potentielles couleurs qui sont détectées pendant la seconde phase.

f - Algorithme:

Ci-dessous, le diagrammes des phases 1 et 2:

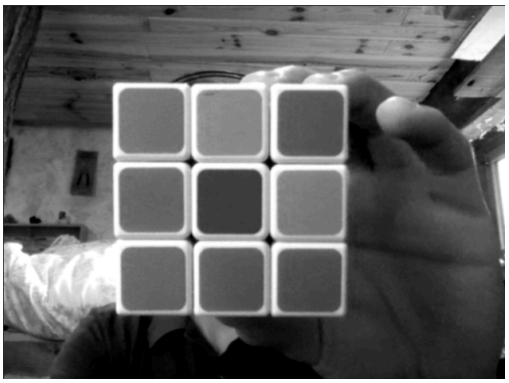


11 - diagramme des phases de détection du cube



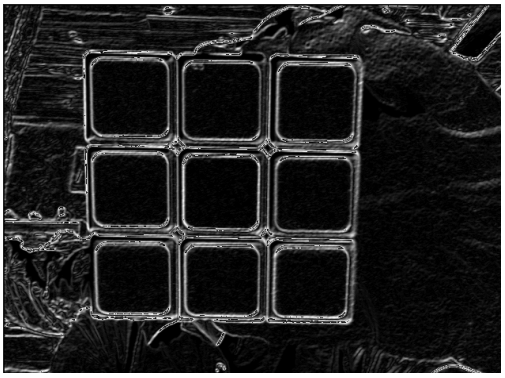
12 - image vierge

Pré-traitement: passage de l'image en niveaux de gris, augmentation des contrastes et application d'un flou Gaussien pour gommer le bruit et faciliter la détection des contours.

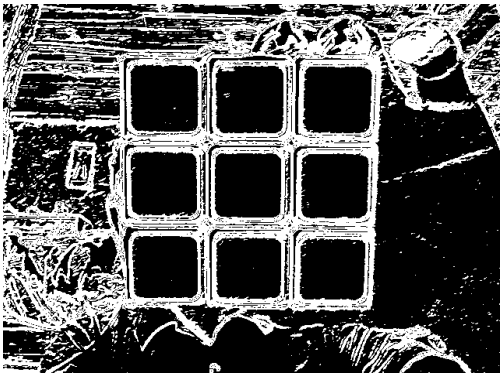


13 - prétraitement

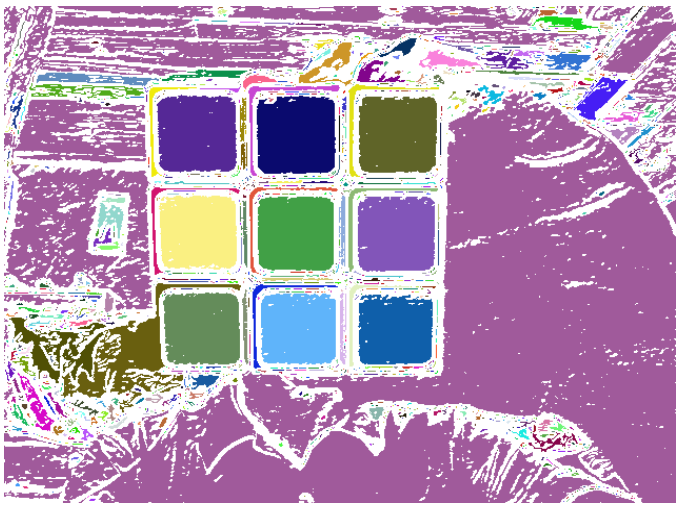
Détection des contours: application d'un filtre convolutif de Sobel permettant de faire ressortir le gradient de chaque pixel puis binarisation de l'image.



14 - détection de contours

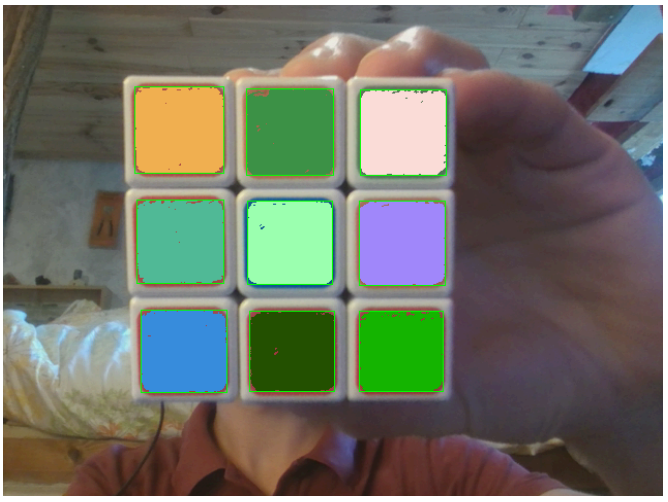


15 - binarisation



16 - extraction de masses

Extraction des masses: application d'un algorithme de détection de masses; on part d'un pixel quelconque puis s'étend dans toutes les directions jusqu'à ce qu'on rencontre les limites de la masse (lorsque les pixels ne sont plus de la même couleur). On recommence jusqu'à avoir couvert toute l'image.



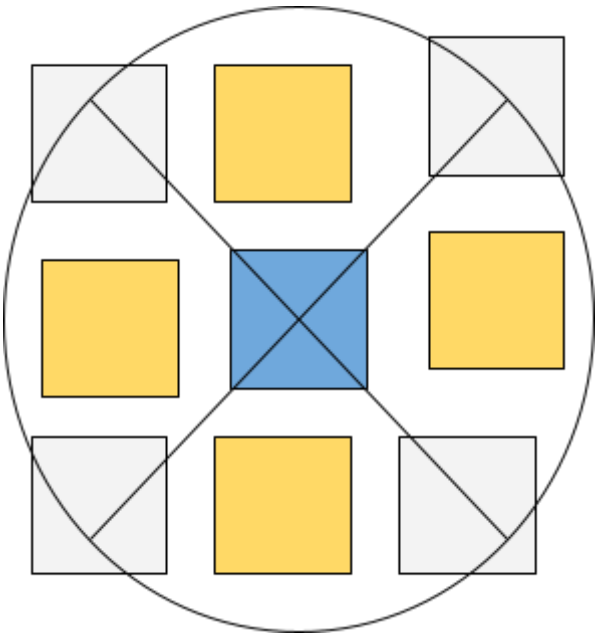
17 - filtrage des masses

Filtrage des masses: calcul des extrêmes des masses, puis des dimensions, de la rotation et du taux de remplissage (rapport entre la taille déduite des dimensions *-h x w-* et la taille réelle). On filtre ensuite en fonction de tous ces paramètres pour ne garder que ce qui est carré.

Calcul des couleurs: pour chaque masse, on fait une moyenne de la couleur de ses pixels (en son centre pour éviter les parasites des bordures).
Dans la phase 2 uniquement, on estime à quelle couleur de la table créée lors de la phase 1 celle-ci correspond en calculant la distance euclidienne avec chacune des entrées de la table pour trouver la plus proche.

Mise à jour de la table des couleurs *(phase 1 uniquement)***:** si une nouvelle couleur est détectée pendant cette itération, elle est ajoutée à la table. Un seuil est défini (modifiable dans les paramètres) de distance minimum que la nouvelle couleur doit avoir avec les autres pour être considérée comme différente.

Création du graphe *(phase 2 uniquement)***:** pour chacune des masses restante, on calcule ses voisins puis on trouve la racine du graphe en le parcourant une première fois.



18 - calcul des voisins

Pour chaque quartier autour de la masse, on récupère la masse la plus proche à l'intérieur de ce dernier.

Mise en correspondance *(phase 2 uniquement)***:** à partir du graphe on associe chaque couleur détectée au Facelet correspondant dans le cube. Si le Facelet en question est un centre, on en profite pour mettre à jour la table de correspondance (on associe alors le tag de la face courante à la couleur en question)

Facelet	RGB
0	Bleu
1	Vert
...	...
54	Jaune

19 - données RGB

RGB	Tag
Bleu	U
Jaune	F
...	...
Vert	L

20 - table de couleurs

Facelet	Tag
0	U
1	L
...	...
54	F

21 - cube final

Une fois cet algorithme effectué et les 6 faces détectées, nous avons notre table de correspondance (ColorTable) et notre tableau des couleurs détectées (RgbData) remplies. On opère alors la transition: pour chaque Facelet de notre RgbData, on récupère son Tag dans la ColorTable et ainsi, nous reconstituons le cube dans notre représentation sous forme de Facelet, utilisable par notre solveur.

g - Limites:

L'algorithme que nous avons créé possède toutefois plusieurs limites. Tout d'abord, lorsqu'une source de lumière vive est présente dans le champ de la caméra, cela influe fortement sur les couleurs détectées. Ensuite, si l'une des cases possède un motif (comme c'est parfois le cas), le programme aura du mal également à comprendre que c'est un carré car son taux de remplissage sera faible. Finalement, nous avons rencontré une difficulté importante à différencier le orange du rouge, surtout quand la luminosité est faible. En effet, au format RGB, ces deux couleurs sont très proches. Pour pallier ce problème, nous avons essayé d'utiliser le format HSV et XYZ pour finalement retenir le format LAB qui est le plus efficace dans notre cas. *(Nous l'utilisons uniquement lors du calcul de distance)*

IV - Simulateur:

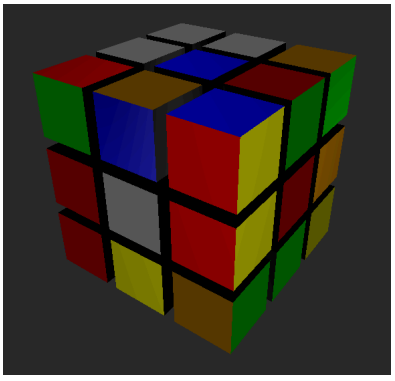
Cette partie est dédiée au simulateur du cube. L'objectif de cette fonctionnalité est de fournir à l'utilisateur un environnement d'expérimentation du Rubik's Cube facile à prendre en main. Notre principale problématique a été de trouver un moyen simple de créer un rendu 3D et de l'intégrer à notre interface GTK.

a - Utilisation:

Le simulateur est la première vue qui apparaît lorsqu'on clique sur le bouton cube dans le menu.

Avant de rentrer dans les détails de son fonctionnement, voici un aperçu de ses composants:

Rendu 3D: au centre de la vue, le cube actuel est affiché à l'utilisateur.



22 - rendu 3D du cube

Mouvements: les 6 boutons en bas de l'écran permettent d'appliquer des mouvements au cube avec une animation. Ce sont les 6 principaux mouvements. Pour accéder à leurs déclinaisons, il suffit d'aller dans le paramètres et de cocher la case *Advanced mode*.



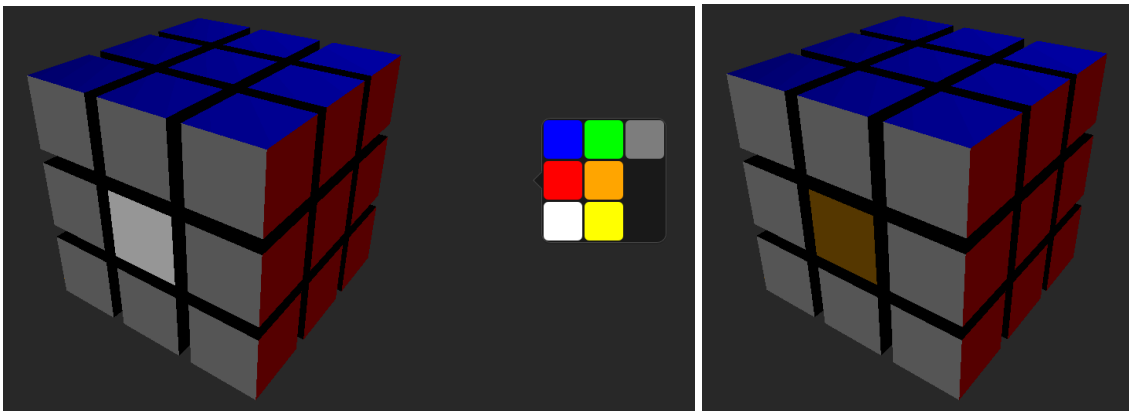
23 - boutons de mouvements

F1	B1	L1	R1	U1	D1
F2	B2	L2	R2	U2	D2
F'	B'	L'	R'	U'	D'

24 - boutons de mouvements avancés

Rotation: en cliquant sur la vue 3D et en déplaçant la souris, la caméra tourne autour du cube permettant de voir ce dernier sous toutes ses faces. Le cube revient à sa position de départ automatiquement.

Modification manuelle: en cliquant directement sur le cube, il est possible de changer la couleur de chacun de ses Facelets. Afin de pouvoir modifier les Facelets sur les faces non visibles du cube, l'utilisateur peut décocher la case *Reset camera* dans les paramètres pour que le cube ne revienne pas à sa rotation de départ automatiquement.



25 - modification du cube



Réinitialisation: le bouton *Reset* permet de réinitialiser l'état du cube.



Mélange: le bouton *Shuffle* mélange le cube avec une animation. Le nombre de mouvements par défaut est 10 mais il peut être réglé dans les paramètres.



Solveur: le bouton *Solver* permet d'envoyer le cube au solveur et de récupérer une solution qui est affichée dans la vue *Solution* si le cube est valide (voir les parties *Solution* et *Solveur*).

b - Ressources:

Afin de créer un rendu 3D, nous utilisons la librairie ***kiss-3d***. Celle-ci nous permet de créer facilement des objets 3D simples mais ne prend pas en compte de détection de collision ni de méthode de rotation simple de la caméra.

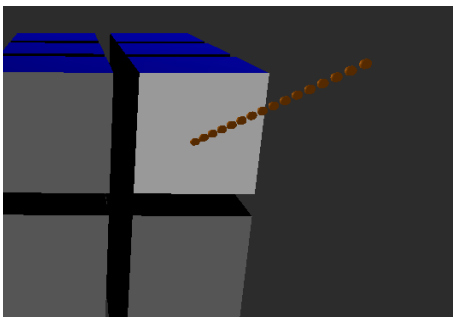
Pour intégrer ce rendu à notre interface GTK, nous capturons ce dernier sous forme de tampon de pixel et écrivons ces octets directement dans une image GTK.

c - Ray-casting:

Kiss-3d n'incluant pas de détection de collision, nous avons dû implémenter le ray-casting pour la modification des Facelets nous-même. Pour ce faire, voici le processus:

- capture de la position de la souris dans la fenêtre GTK
- conversion en coordonnées dans le rendu kiss-3d
- envoi d'un rayon de collision à partir des coordonnées et dans la direction de la caméra (on définit des points à intervalles réguliers sur cet axe)
- détection des collisions entre chacun des points du rayon et toutes les faces visibles du cube

Pour détecter une collision entre un Facelet et un des points du rayon, nous sommes donc obligés de récupérer directement les sommets de ce Facelet.



25 - points testés lors du ray-casting

d - Rotation:

Comme dit plus haut, kiss-3d n'offre pas de moyen simple de faire tourner la caméra autour du cube, il nous a donc fallu recoder entièrement ce système de mouvement en partant des évènements de la souris que GTK détecte.

V - Solveur:

Cette partie est dédiée à la résolution du cube. L'objectif de cette fonctionnalité est de fournir à l'utilisateur une solution à son cube en un minimum de coups et un minimum de temps.

a - Choix de l'algorithme:

Initialement, nous avions prévu de commencer par implémenter l'algorithme de Thistlewaite puis, si cette première tâche était remplie avec succès, tenter d'implémenter l'algorithme de Kociemba. Le second étant une version améliorée du premier, les performances en sont aussi augmentées. L'algorithme de Thistlewaite renvoie une solution avec une moyenne d'environ 50 coups quand celle de Kociemba est d'approximativement 20 coups.

b - Fonctionnement:

La notion d'ensemble de cubes déterminé par les mouvements nécessaire à la résolution de tous ses éléments est la pierre angulaire de l'algorithme de Thistlewaite. Ce dernier utilise les sous-ensembles suivants:

Etape	Sous-ensemble	Description	Mouvements autorisés
1	G0	Ensemble de toutes les configurations possibles du Rubik's Cube	<U, D, L, R, F, B>
2	G1	Toutes les arêtes sont orientées correctement	<U, D, L2, R2, F2, B2>
3	G2	Arêtes orientées correctement et arêtes de la tranche centrale (M-slice) positionnées dans cette tranche	<U, D, L2, R2, F2, B2>
4	G3	Arêtes et coins positionnés pour être résolus avec des mouvements restreints	<U2, D2, L2, R2, F2, B2>
5	G4	État résolu du cube	Résolu

L'objectif de Thistlewaite est de réussir à faire passer le cube à résoudre du groupe G0 au groupe G4, en utilisant les groupes intermédiaires G1, G2, et G3.

L'algorithme de Kociemba utilise également cette notion de sous-ensemble. Ce dernier repose sur le principe suivant: si l'on tourne les faces d'un cube résolu sans utiliser les mouvements R, R', L, L', F, F', B et B', on génère un sous-ensemble de cubes appelé G1 = <U, D, R2, L2, F2, B2>. Dans ce sous-ensemble, les orientations des coins et des arêtes restent constantes, et les quatre arêtes entre les faces U et D restent dans cette tranche.

Alors que Thistlewaite a composé son algorithme de 4 phases majeures, Kociemba à augmenté la complexité de ces phases pour en diminuer le nombre à 2 seulement.

Dans la phase 1, l'algorithme cherche des séquences pour transformer un cube mélangé en un cube du sous-ensemble G1. Il utilise un algorithme de recherche appelé A* itératif avec une fonction heuristique qui ne surestime jamais le nombre de mouvements nécessaires pour atteindre l'état cible. Cette heuristique, basée sur une table de consultation en mémoire (*pruning table* dans le code), permet d'élaguer les manœuvres jusqu'à 12 mouvements à l'avance.

La phase 2 restaure le cube en utilisant uniquement les mouvements du sous-ensemble G1. Elle rétablit la permutation des 8 coins, des 8 arêtes des faces U et D, et des 4 arêtes de la tranche UD. La fonction heuristique de cette phase estime le nombre de mouvements nécessaires pour atteindre l'état cible.

L'algorithme ne s'arrête pas à la première solution trouvée, mais continue à chercher des solutions plus courtes en calculant de nouveau la phase 2 à partir de solutions sous-optimales de la phase 1. Par exemple, une première solution pourrait nécessiter 10 mouvements en phase 1 et 12 en phase 2, tandis qu'une solution ultérieure pourrait en nécessiter 11 en phase 1 et seulement 5 en phase 2. L'algorithme continue ainsi jusqu'à trouver une solution dont la longueur est inférieure au paramètre *max length* ou jusqu'à dépasser le *timeout*.

Dans notre implémentation, l'algorithme en deux phases ne cherche pas des solutions globalement optimales qui traversent les phases 1 et 2, ce qui améliore considérablement la vitesse. Kociemba propose également une méthode pour trouver une solution optimale au cube mais celle-ci prend beaucoup plus de temps.

c - Les tables:

L'algorithme de Thistlewaite implique la création de 4 tables de transition où la table n permet de passer de Gn à Gn+1. Le parcours de ces tables est linéaire car elles stockent simplement pour chaque permutation du sous-ensemble une solution de transition pour passer au sous-ensemble suivant. Leur exploitation est donc relativement facile mais leur création est grandement compliquée par des limitations matérielles. En effet, dû au nombre de permutations pour lesquelles il faille chercher une solution, leur création demande énormément de temps. Pour nous, 11 ans avant optimisation (d'après nos estimations) et 1h30 après.

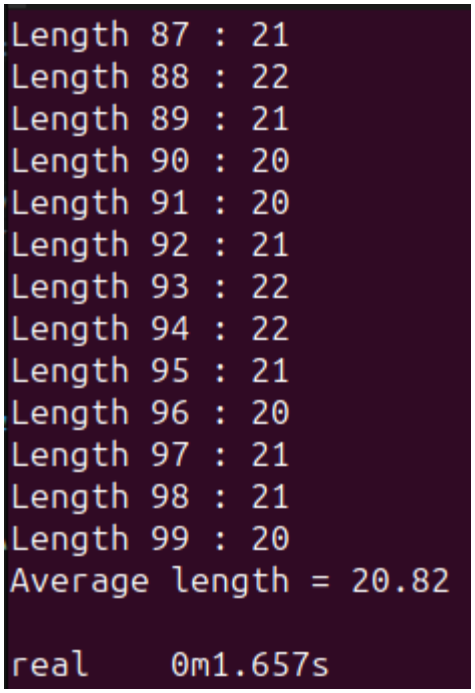
L'algorithme de Kociemba, quant à lui, possède 19 tables plus ou moins volumineuses. Il en existe 2 principaux types:

- les tables de *pruning* (*phas1_prun, phase2_prun)* qui contiennent les heuristiques pour le parcours A* itératif
- les tables de symétries (*co_sym, fs_sym* ...) et de conjonction (*conj_twist, conj_flip* ...) qui permettent de trouver des équivalences entre les cubes pour faciliter les parcours

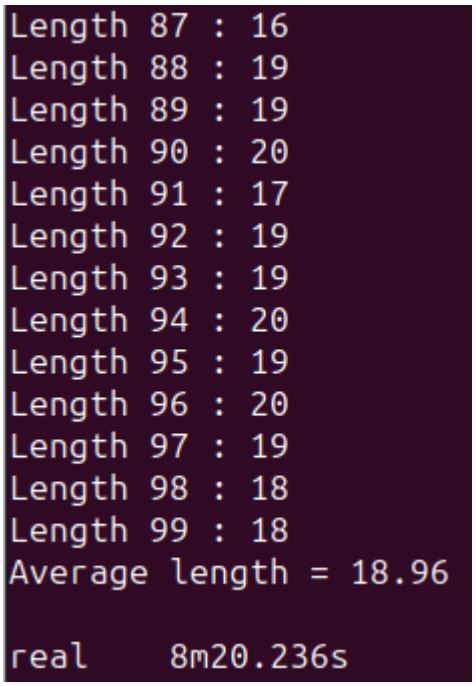
A titre indicatif, bien qu'il y ait 5 fois plus de tables dans notre nouveau solveur, ces dernières font environ 75 Mo pour 650 Mo avec l'ancien.

c - Performances:

Notre implémentation de cet algorithme fournit des résultats plus que satisfaisants.



26 - première solution



27 - plus courte solution

Nous avons mélangé 100 cubes avec 100 mouvements aléatoire chacun puis les avons résolu.

Pour le premier test (table de gauche), nous avons mis le paramètre *max length* du solveur à u32::MAX. Dès lors, le solveur s'arrêtera à la première solution qu'il trouvera. En l'occurrence, la longueur moyenne de la solution sur les 100 cubes est de **20.82 mouvements** et le test tourne en **1,6 secondes.**

Pour le second test (table de droite), nous avons mis *max length* à 1. Ainsi, le solveur ne s'arrêtera que lorsqu'il aura trouvé une solution dont la longueur est inférieure à 1, ce qui n'arrivera jamais. Il est donc nécessaire de lui fournir également le paramètre *timeout* que nous avons ici initialisé à 5 secondes. En paramétrant ainsi le solveur, ce dernier cherchera pour chaque cube pendant cinq secondes et renverra la solution la plus courte qu'il aie trouvé. En l'occurrence, la longueur moyenne de la solution pour les 100 cubes est de **18,96 mouvements** et le test tourne en **8 minutes 20** (environ 500 secondes).

Pour bien comprendre ces résultats, il est important de rappeler que la longueur moyenne de solution optimale sur l'ensemble des cubes possibles est de **18,63 mouvements.**

VI - Solution:

Cette partie est dédiée à l'affichage de la solution trouvée par le solveur. L'objectif de cette fonctionnalité est de fournir à l'utilisateur un moyen intuitif de visualiser cette solution et de la parcourir facilement.

a - Utilisation:

La solution du cube est présentée sous forme de "timeline" avec un rendu 3D. Ainsi, l'utilisateur pourra suivre directement l'évolution du cube.

 **Suivant:** montrer le mouvement suivant.

 **Précédent:** montrer le mouvement précédent.

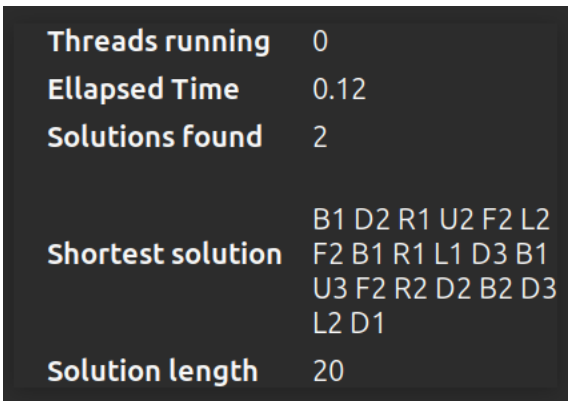
 **Aller au début:** revenir au début de la solution.

 **Défiler:** faire défiler automatiquement la solution jusqu'à la fin (le défilement peut être arrêté en cliquant sur un des autres boutons).



Mouvement: indique le nom du prochain mouvement à effectuer.

Timeline: indique l'avancement actuel de la solution.



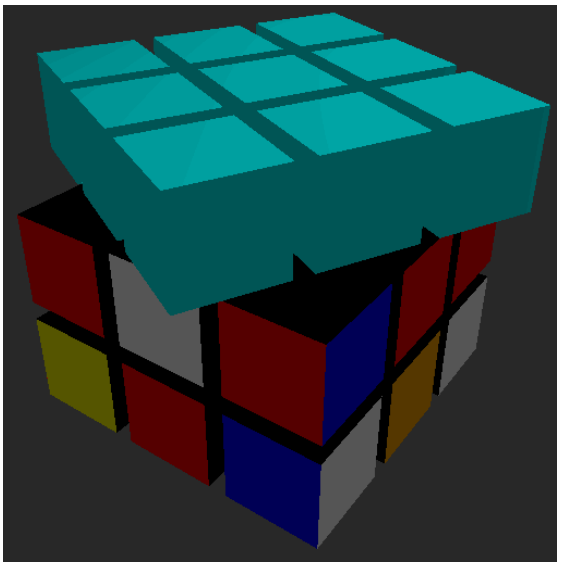
28 - panel d'informations

Informations: affiche diverses informations sur la solution et le solveur.

- le nombre de threads en cours
- le temps de recherche
- le nombre de solution trouvées
- la solution trouvée la plus courte
- la taille de la solution

b - Ergonomie:

Sachant que l'utilisateur pourrait ne pas connaître les noms des mouvements et dans l'optique de simplifier la compréhension de la solution, nous avons ajouté la prévisualisation du mouvement suivant. Ainsi, la tranche concernée est colorée en bleue et le début du mouvement est initié pour indiquer dans quel sens la tourner.



29 - prévisualisation d'un mouvement du cube

VII - Interface:

Cette partie est dédiée à l'interface utilisateur dans sa globalité. L'objectif de cette interface est de permettre à l'utilisateur de naviguer efficacement et simplement dans le logiciel. Il est également important que cette dernière soit intuitive pour que l'utilisateur ne soit pas perdu lors de ses débuts.

a - Ressources:

L'entièreté de l'interface a été créée avec ***GTK-3***. Pour la mise en place et l'agencement, nous avons utilisé ***Glade*** puis nous avons connecté tous les signaux directement depuis le code.

b - Organisation:

WidgetPool: Travailler avec GTK peut rapidement produire une grande quantité de code répétitif, notamment la récupération des différents widgets auxquels on souhaite accéder.

```
// Récupère le label
let label: gtk::Label = builder
    .object("label")
    .expect("Failed to get label");
// Assigne le signal
label.connect_signal(...)

// Récupère le bouton
let button: gtk::Button = builder
    .object("button")
    .expect("Failed to get button");
// Assigne le signal
button.connect_signal(...)
```

30 - exemple de code de récupération des widgets

C'est pourquoi, nous avons créé une structure **WidgetPool**, qui nous permet de réduire grandement ce code.

```
1 // Récupère le label
2 let label: gtk::Label = widget_pool.get("label")
3 // Assigne le signal
4 label.connect_signal(...)
5
6 // Récupère le bouton et assigne le signal
7 widget_pool.get_button("button").connect_signal(...)
```

31 - exemple de code optimisé avec un WidgetPool

Comme ce sont des opérations réalisées très régulièrement, cela nous permet de réduire grandement la taille du code de connexion des signaux et d'augmenter sa lisibilité.

Parameters: En Rust, les variables globales n'existent pas. Or il nous faut pouvoir accéder depuis n'importe où dans l'arborescence de notre projet au paramètres variables qui sont modifiés par l'utilisateur dans l'onglet paramètre.

Pour ce faire, nous utilisons un **singleton** combiné avec la librairie ***lazy static***, placé à la racine de notre projet.

```
1 lazy_static! {
2     pub static ref SINGLETON: Arc<Mutex<Parameters>>
  = Arc::new(Mutex::new(Parameters::new()));
3 }
```

32 - définition du singleton des paramètres

c - Ergonomie:

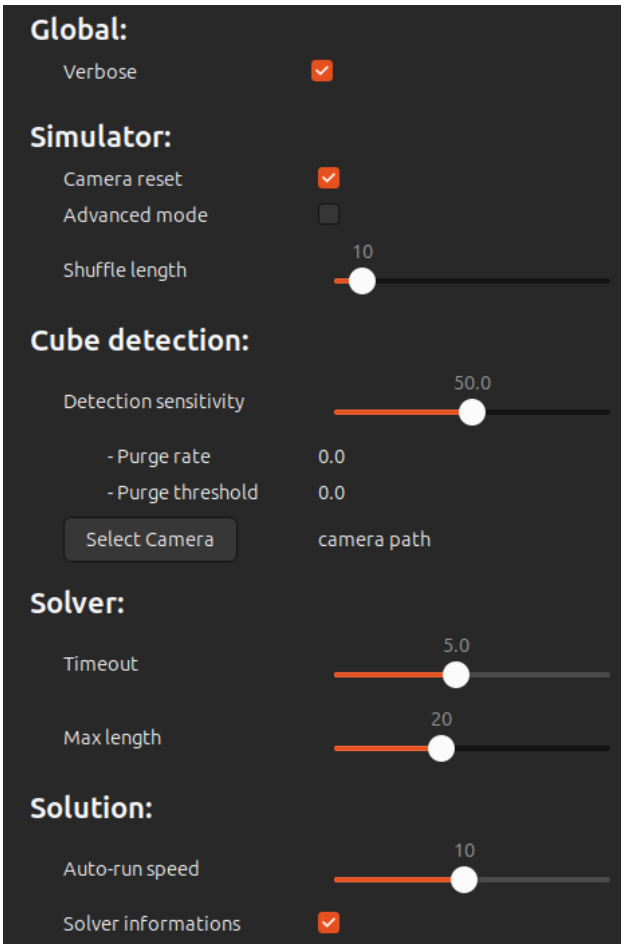
Afin d'aider l'utilisateur dans la navigation, dans la possibilité ou les images des boutons ne seraient pas assez explicites, des *tooltips* ont été ajoutés.



33 - *tooltips* des boutons

d - Paramètres:

Un utilisateur lambda trouvera dans Rubik's 2000 un logiciel pré paramétré et calibré pour fonctionner efficacement que ce soit pour le solveur, la caméra ou le simulateur. Mais nous avons également laissé la possibilité à un utilisateur plus avancé souhaitant obtenir plus de contrôle sur son expérience, la possibilité d'agir sur de nombreux paramètres.



34 - panel des paramètres

Global:

- *verbose* : s'il faut afficher les messages informatifs et les *warnings* dans la console.

Simulator:

- *camera reset* : si la rotation de la caméra doit être réinitialisée après un mouvement (utilisé pour modifier les faces non-visibles du cube)
- *advanced mode* : d'il faut afficher les 8 boutons dérivés des 6 mouvements initiaux
- *shuffle length* : le nombre de mouvements utilisés pour mélanger le cube

Cube detection:

- *detection sensitivity* : sensibilité de la détection du cube. A titre indicatif, les deux paramètres modifiés sont affichés en dessous.
  Attention! : Un algorithme trop sensible captera des données parasites, tandis qu'un trop peu sensible n'en captera pas assez.
- *select camera* : ouvre une fenêtre pour choisir la caméra à utiliser

Solver:

- *timeout*: temps écoulé à partir duquel le solver arrête sa recherche même s'il n'a pas trouvé de solution.
- *max length*: longueur de solution à partir de laquelle le solveur s'arrête.

Solution:

- *auto-run speed*: la vitesse de défilement de la solution.
- *solver informations*: si le panel d'informations sur la résolution du cube doit être affiché.

VIII - Site Web:

Le site web a été mis à jour depuis la dernière soutenance sur l'avancement général. A part cela, nous l'avons laissé tel quel, déjà satisfait du résultat lors de la dernière soutenance.

VIII - Conclusion:

Après de longues heures de travail, nous sommes heureux de constater l'accomplissement de notre projet. Depuis le deuxième soutenance, de nombreux ajout ont été fait:

- rotation de la caméra
- ray-casting
- détection dynamique du cube
- affichage de la solution
- paramètres
- mouvements avancés
- solveur (Thistlewaite puis Kociemba)

Tout cela mis bout à bout nous a permis d'acquérir de solides bases en rust et d'être fier de notre logiciel, Rubik's 2000.

Quelques chiffres:

Pour conclure Rubik's 2000 c'est:

- **10 712** lignes (dont **9 387** de code)
- **1** cube pour **43 252 003 274 489 856 000** possibilités
- **3** likes sur github
- **4/4** utilisateurs satisfaits
- **24** modules, **276** fonctions, **18** structures